THIRD REVISION · IMPLEMENTATION-READY DESIGN

Radix-2 Decimation-in-Time FFT

Interactive demo proposal for svg-dom-graph · Revision 3

Architecture unchanged; final details settled
Use **one single-complex data node per lane per stage boundary** and real **Scene::add_edge_with** connectors. The stage toolbar navigates a hidden L-cell driver; a separate pair selector inspects the butterfly. Revision 3 makes validation, total counts, accessibility, controls, routing review and rollback cache explicit. No new core API.

Revision 3: remaining review items closed

ITEM	NORMATIVE RESOLUTION
1 · N = 1	Reject any native length smaller than 2 before calculating bit-reversal indices; return an error rather than shifting by word size.
2 · step driver	Report 32/80 FFT lane nodes and 33/81 total overview Scene nodes including the hidden driver; exclude the driver from bounds.
3 · accessibility	Connectors carry relationships through endpoint node names and are not intended as tab stops; test the browser accessibility tree at N=16.
4 · presets	Native N and preset selects; impulse x[1], unit constant, real cosine at bin 1, deterministic mixed complex. Zero vector is a native test fixture.
5 · routing	Manual N=16 final-stage screenshot inspection is a release gate; widen inter-stage spacing when arrow or line routing is unclear.
6 · semantics	Distinct unstarted announcement; last_committed_pair is the committed stage per-stage pair cache entry.

Scope. Forward, unnormalised radix-2 Cooley–Tukey DIT FFT. The teaching UI supports N=8 and N=16 (native arithmetic tests also N=2,4,32); complex64 values; stage-by-stage stepping plus a separate butterfly inspector. Reverse FFT, arbitrary length, freeform editing and public FFT APIs remain out of scope.

Compatibility. The algorithm computes in `demo_common::complex::ComplexBuf`; SVG nodes only visualise caller-computed values. The design is pinned to repository tree `d6cc290b180a43f8a8e30b4d7493941a7416671a`, crate version 0.2.44, inspected 27 October 2026.

1. Numerical contract

For **$N = 2^L$**, the forward transform is **$X[k] = \Sigma\, x[n]\, \exp(-2\pi i k n/N)$**. The sign is negative and the forward transform is not scaled by 1/N. The result appears in natural k order; only the *input* is bit-reversed.

Bit reversal and stages

```
S0[i] = x[bit_reverse_L(i)]
for s = 1..L:
    m = 2^s; half = m/2
    for start in (0..N).step_by(m):
        for j = 0..half:
            u = start+j; v = u+half
            w = exp(-2*pi*i*j/m)
            (S_s[u], S_s[v]) = (a+w*b, a-w*b)
```

Every stage uses its predecessor S(s−1); there are **N/2 disjoint butterfly pairs** at each stage, and each pair contributes four directed data-dependency edges between adjacent stage columns: a→sum, a→difference, b→sum and b→difference. A twiddle factor is a scalar operand of the selected operation, not an additional overview lane.

N	STAGES	OVERVIEW NODES	BUTTERFLY PAIRS	DEPENDENCY EDGES
8	3	32 FFT + 1 driver = **33**	3 × 4 = **12**	12 × 4 = **48**
16	4	80 FFT + 1 driver = **81**	4 × 8 = **32**	32 × 4 = **128**

Stable conventions

- Use `ComplexBuf::unit_roots(m, half)`, which returns positive-angle roots; call `conj()` once for forward twiddles, before processing a stage.
- Call `data.butterfly(start, half, &twiddles)` **once per complete contiguous block**. Do not call it per individual pair.
- Reject native **N < 2**, non-power-of-two lengths and non-finite components *before* computing a reverse-bits shift. Reject UI N outside {8,16}. A length-1 transform is deliberately unsupported; report a typed validation error, never panic.
- The N=8 impulse x[1]=1 must yield S0 input indices [0,4,2,6,1,5,3,7], then final X[k]=exp(−2πik/8). No output permutation or forward 1/N normalisation.

2. Pure algorithm and trace

Implement a DOM-free module at `demo-app/src/fft/algorithm.rs` (prefer private helper types until needed elsewhere). Use `Complex` for scalar trace fields and `ComplexBuf` for full-buffer transforms. Mirror the already-tested FFT loop in `demo-common/src/complex/unit_tests/buffer.rs`.

```
struct PairTrace {
  stage: usize, start: usize, j: usize,
  upper: usize, lower: usize,
  a: Complex, b: Complex, w: Complex,
  product: Complex, sum: Complex, difference: Complex,
}
struct FftTrace {
  input: Vec<Complex>, bit_reversed: Vec<usize>,
  states: Vec<Vec<Complex>>,  // S0..SL
  pairs: Vec<Vec<PairTrace>>, // stage -> start-major, j-minor
}
```

One-pass permutation and pair recording

```
if n < 2 || !n.is_power_of_two() { return Err(BadLength(n)); }
// Only now: bits is in 1..usize::BITS
let order: Vec<usize> = (0..n)
  .map(|i| i.reverse_bits() >> (usize::BITS - bits))
  .collect();
let reordered: Vec<Complex> = order.iter().map(|&i| input[i]).collect();
let mut data = ComplexBuf::from_complex(&reordered);
states.push(reordered);
for stage in 0..bits {
  let half = 1usize << stage; let m = 2 * half;
  let mut tw = ComplexBuf::unit_roots(m, half)?;
  tw.conj();
  let before = data.to_complex();
  for start in (0..n).step_by(m) {
    // Snapshot a,b,w and scalar product=w*b before mutation.
    // Derive scalar sum,difference; append ordered PairTrace.
    data.butterfly(start, half, &tw); // whole block, once
  }
  let after = data.to_complex();
  // Verify each recorded sum/difference maps to after[u/v].
  states.push(after);
}
```

Single ownership of numerical truth. Compute `PairTrace.product = w * b` and `(sum, difference) = a.sum_and_difference(product)` with the existing scalar complex operations, using saved *pre-block* values. The buffer is the transform authority. Independently assert that pair outputs agree with the buffer's post-block lanes to floating-point tolerance. No scalar operation runs inside an SVG node.

Validate N ≥ 2 and N a power of two before `reverse_bits() >> (usize::BITS − bits)`. Collect bit-reversal indices once into `order`; use that stored vector for trace labels and reordered samples. Tests must reject lengths 0, 1 and 3 without panic.

3. Overview rendering: real graph edges

Replace former §5.1 and §7.1: each boundary S0…SL owns **N separate named complex data nodes**, exactly one complex value per node. Each two adjacent boundaries are joined by **four Scene edges per butterfly pair**. All nodes and edges belong to the overview Scene and therefore share its existing pan/zoom transform.

ELEMENT	IMPLEMENTATION CONTRACT
Lane nodes	`Scene::add_named_data_node`, one `NodeValues::Complex64(vec![z])` per lane; names include boundary, lane index and S0 sample index.
Edges	`Scene::add_edge_with`; store all four returned `EdgeId` values in each `OverviewPair`.
Anchors	Prefer outgoing East / incoming West with distinct input/output fractions (such as ¼ and ¾). Routing is exercised in CDP; avoid arrow overlap and crossing node text.
Selection	Use `Scene::set_focus` on the selected four lane nodes and `Scene::set_edge_focus` on its four corresponding directed edges. Both are public.
Values	Completed stages show stored values. Uncompleted stages contain a zero placeholder marked with `set_unreached_cells(id, &[0])`; do not expose future results as already computed.
Layout	Columns use library-measured node rectangles; each row has one stable vertical anchor across boundaries. Keep enough horizontal space for four-edge routing and arrowheads.

Topology and sizes

The FFT topology is immutable for a fixed N and preset. It has **32 FFT nodes/48 edges at N=8** and **80 FFT nodes/128 edges at N=16**, plus **one** hidden step-driver node in the same overview Scene: **33** or **81** total. A typical intermediate lane has in-degree 2 and out-degree 2. Draw all edges once when creating the scene; navigation changes data, markers and focus, not topology.

The nodes in S0 are labelled both *lane i* and *source x[bit_reverse(i)]*. Later columns use *Ss[i]*. Keep names stable while the associated complex values change. Browser tests must verify no duplicate connections and no incorrect directed source/destination.

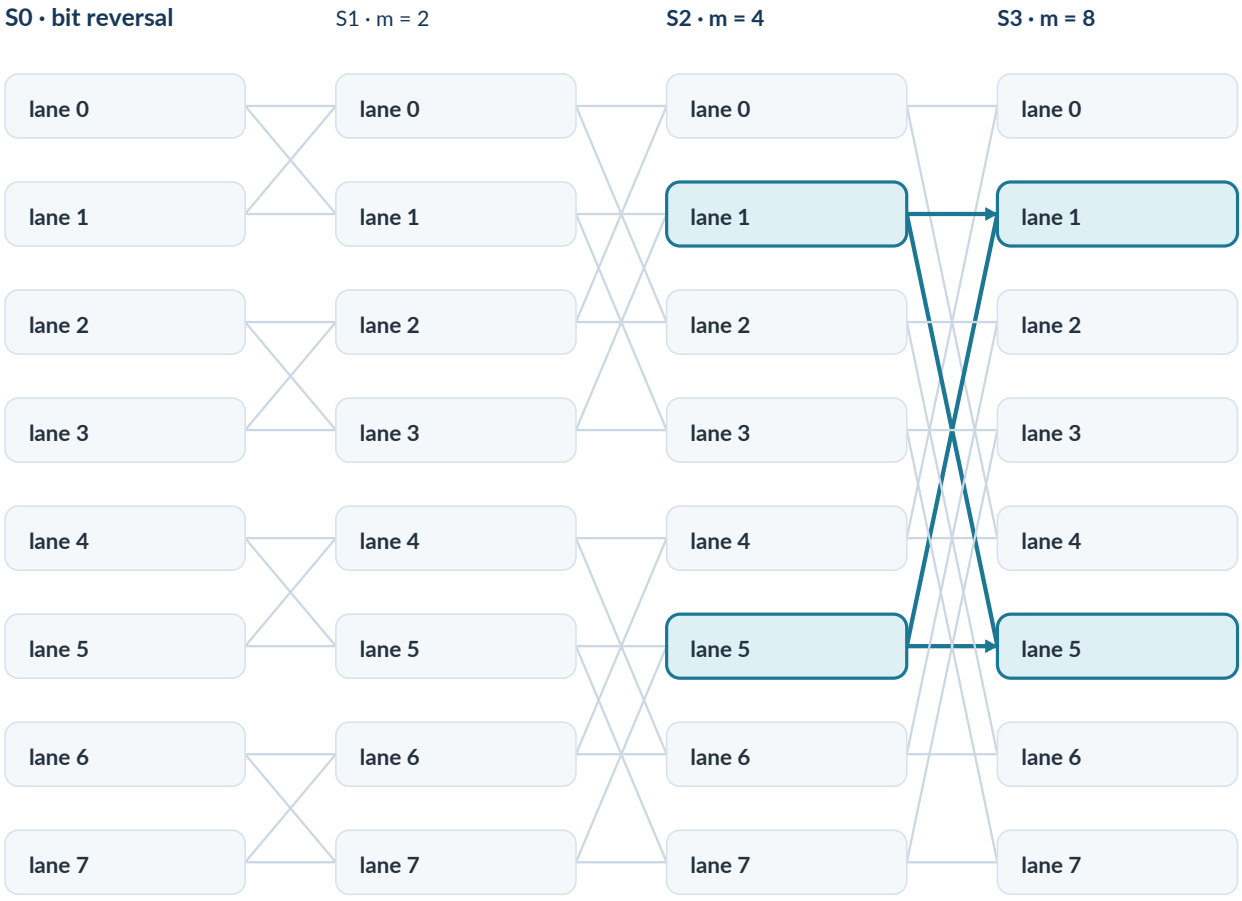
No private-DOM dependency

Do not append custom paths to `.svg-dom-graph-content`, rely on its private implementation, or simulate a second floating wiring layer. `Scene::cell_rect` exists but is unnecessary for these node-to-node connectors; there is no conditional single-node fallback. Use the public edges as the one supported implementation for both N values.

Fit and count rule. The hidden driver is positioned at (−10000, −10000); it is not part of the FFT layout. Size/fit the canvas from the measured rectangles of the explicit (L+1)×N visible lane nodes, connecting paths and margins, *not* from all Scene nodes or the SVG content group bounding box. CDP tests count FFT data nodes by stable lane names or recorded NodeIds and separately assert the one hidden driver; manually check Reset zoom / fit-to-view does not jump to the off-screen driver.

4. Eight-point stage-flow schematic

A schematic of the actual graph topology. Every drawn route corresponds to an overview edge created with `Scene::add_edge_with`. The selected stage-3 pair combines lanes 1 and 5; the four darker routes are the edges focused with `set_edge_focus`.



x indices: 0, 4, 2, 6, 1, 5, 3, 7
Highlighted routes: the four dependencies of stage 3, pair (lanes 1, 5).

Figure 1. Four boundaries × eight lanes; 48 real dependency edges in total. Rectangles stand for individual complex data nodes, not cells of one data grid. The diagram is conceptual: the library decides the actual routed SVG paths and arrowhead positions.

Pair-index mapping

```
pairs(stage) = [
  (start+j, start+j+half, j, start)
  for start in (0..N).step_by(2*half)
  for j in 0..half
]
```

The pair selector is **one-based in its UI labels** but **zero-based in trace storage**. For stage 3 at N=8, selecting pair 2 (index 1) yields (upper, lower)=(1,5), j=1, and $W_8^1=(1-i)/\sqrt{2}$. The same ordered enumeration builds edges, identifies inspector values and drives tests.

5. Inspector and sizing lifecycle

A separate inspector SVG contains one `Scene::add_butterfly_node` instance fed by three single-complex operand nodes and linked to two output nodes, matching the existing `demo-app/src/butterfly.rs` example. The Butterfly API renders multiplication, addition and subtraction; the demo supplies every numerical result.

For one trace selection

LOCATION	VALUE
Operand a	`PairTrace.a` from previous-stage upper lane
Operand b	`PairTrace.b` from previous-stage lower lane
Twiddle w	`PairTrace.w` = W_m^j, negative exponent
Product node	`PairTrace.product = w*b` (scalar Complex multiplication)
Add/sub outputs	`PairTrace.sum` / `PairTrace.difference`

Exactly defined initial and reset selection

For any displayed stage, the default pair is the **first pair in start-major, j-minor order whose a or b operand is mathematically nonzero** (a.re≠0 or a.im≠0 or b.re≠0 or b.im≠0). If no such pair exists, select **pair index 0**. Zero includes signed floating zero. This handles the all-zero fixture deterministically. For constant input, stage 1 begins with pair 0.

When stage is unstarted (`None`), the inspector is clearly titled **"Preview: stage 1, pair p"**, using that rule. It can show its calculated result as a preview, while S1…SL overview nodes remain not-yet-computed. Restart keeps the same preset, clears visited-pair history and returns to the same preview rule.

Size once, then update

Because `Scene::set_data_values` does not resize nodes, enumerate the *reachable trace values* for every overview lane and inspector slot for the selected (N, preset). Use library measurement, not string length alone, to choose a **widest candidate** for each node. Construct each node with its widest candidate, then immediately write its initial placeholder or selected value. This is the existing Butterfly demo's sizing pattern applied systematically.

For **pair changes**, update the same inspector scene and Butterfly node in place. For **N or preset changes**, rebuild as necessary; before replacement capture `scene.node_rect(butterfly.frame)?.origin` and the inspector view. After creating the replacement, call `move_butterfly(&butterfly, saved_origin)` to preserve the dragged frame position, then restore the view. If a new canvas cannot fit the saved origin, constrain it to the valid frame area and disclose that adjustment in the UI.

6. Stage and pair navigation

The overview is governed by one hidden numbered step-driver **data node**, created with `demo_common::util::add_step_driver(&scene, OVERVIEW_ID, L)` (internally a one-row U16 array). The `Scene::show_selection_toolbar(driver, ..., on_step)` attaches to *that driver only*, never to a sample lane node.

DRIVER SELECTION	SEMANTICS	INSPECTOR
`Selection::None`	Unstarted; S0 available; S1…SL unreached.	Explicit preview of stage 1; default or user-chosen pair.
`Selection::Cell(i)`	Exactly stages 1…i+1 have completed (i=0…L−1).	Selected pair belongs to stage i+1.
Rebuild	New L-cell driver + toolbar starts as None; paint the unstarted state before activating it.	Default preview; cached pair choices cleared.

Deterministic pair policy

- Maintain a per-stage successful-selection cache `last_pair[stage]` (one-based stage index 1..L, zero-based pair index). The term `last_committed_pair` in failure recovery means `last_pair[last_committed_stage]` when started, or the cached/derived stage-1 preview pair when unstarted; otherwise use the first-informative rule. Pair changes never run the numerical kernel again.

- On Next/Previous, read `transition.to`, determine the new completed-stage count, then repaint from immutable `FftTrace`. `None` from either Prev-at-first or Restart means the same unstarted state.

- For each successful stage navigation, update completed/unreached lane values, move the four edge-focus marks, update the inspector and change one persistent status note and live announcement.

- The toolbar is authoritative for its *own* current selection. A separate `last_committed_stage` is retained only for error recovery and consistency checks; it never drives ordinary stepping independently.

Disagreement rule

After a toolbar callback failure, compare subsequent `transition.from` against the last committed driver position. If they disagree, do **not** apply the new transition or skip a stage: first restore the driver to the committed selection and repaint the committed state. Rebuilds set both the new driver and runtime cursor to `None`; never copy old NodeIds into the replacement scene.

7. Updates, rollback and teardown

Multi-node paints are not atomic. Implement one `render_snapshot(trace, stage, pair)` function that writes every displayed lane value and unreached flag plus all selected focus states and inspector values. Re-invocation with the same arguments must be safe and able to repair partially applied DOM updates.

Toolbar callback: the rollback sequence

```
on_step(scene, driver, transition):
  let before = runtime.last_committed_stage;
  let wanted = stage_from(transition.to);
  if transition.from != expected(before) { repair(before); return; }
  match render_snapshot(wanted, choose_pair(wanted)) {
    Ok(()) => commit(wanted),
    Err(err) => {
      report_error(err);
      // scene was already moved by the toolbar:
      try_set_selection(driver, selection_of(before));
      try_render_snapshot(before, pair_for_committed_stage(before));
      // remain pending repair if either restoration fails
    }
  }
```

Here `pair_for_committed_stage(before)` reads the stage-indexed cache (`last_committed_pair`), defaulting to the defined first-informative stage-1 preview if `before=None`. The callback can safely call `Scene::set_selection`: the current library releases its Scene borrow before invoking `on_step`, and programmatic `set_selection` does not generate another toolbar callback. On a successful restore, the toolbar itself returns to the last committed stage.

If rollback also fails: keep the last committed logical state, show a visible “display needs repair” error, and expose a labelled **Retry display** HTML control. While recovery is outstanding, reject new stage/pair transitions rather than accepting an unverified cursor. The retry reapplies the last committed snapshot, then explicitly sets the step-driver selection to the corresponding value; only a successful repair clears the banner.

Pair input and rebuild failures

- When an HTML pair selector moves before a failing render, restore its displayed value to the last committed pair, attempt to repaint the old pair, and keep status text honest. Pair cache is committed only after all dependent values and focus markers succeed.

- For N/preset changes: validate and compute the new trace first; build replacement scenes and their controls in a measurable staging host; preserve old scene/view/inspector origin/focused control until replacement is ready. Only then replace the visible host, restore focus, and retire the old scenes/listeners.

- On teardown, drop callbacks tied to superseded scenes; ignore queued input events carrying stale scene generation/NodeIds. An unsuccessful staging build leaves the old live scene usable.

8. Accessibility and user controls

Connector exposure is deliberate

An overview at N=16 has **80 named FFT lane nodes, one hidden driver and 128 connector paths**. `Scene::add_edge_with` creates SVG `<path>` elements without an explicit accessible name, role, or `tabindex`; it also appends "Output to …" and "Input from …" relationships to each *endpoint node* accessible description. Thus the data nodes, not 128 additional named edge controls, are the primary reading surface.

- Do **not** make connector paths independent tab stops or add repetitive edge labels in the demo. Use existing endpoint relations to expose the topology; the selected butterfly is described in a concise persistent status paragraph and aria-live announcement.

- `set_edge_focus` adds visual emphasis and `aria-current` to the four selected paths. Do not rely on path aria-current being announced: speak selected stage, pair, lanes, twiddle and product in status text.

- In CDP at N=16, inspect the real browser accessibility tree and tab order: exactly the intended native selects, stepping toolbar and zoom controls are controls; path elements must not create 128 named/focusable items. If a browser exposes them that way, stop the release and resolve accessibility intentionally rather than querying private SVG internals.

Native HTML controls and their values

CONTROL	CHOICES / MEANING
Length `#fft-length`	Native labelled `<select>` with only 8 and 16. Changes validate/compute, then rebuild the overview and inspector.
Preset `#fft-preset`	Native labelled `<select>`: Impulse at x[1]; Unit constant; Cosine (bin 1); Mixed complex. Rebuild and restart.
Pair `#fft-pair`	Native labelled `<select>` with N/2 items labelled "Pair 1 … N/2" and lanes (u,v); trace index stays 0-based.
Stage buttons	Existing `show_selection_toolbar` on hidden driver: Prev, Next, Restart. Toolbar position alone determines completed stages.

Explicit preset equations, N ∈{8,16}

Impulse: `x[1]=1`, others 0. Constant: `x[n]=1+0i` for every n. Bin-centred real cosine: `x[n]=cos(2πn/N)+0i` (bin **1**, plus conjugate bin **N−1**; unnormalised spectral peaks N/2 each, other bins 0 to rounding). Mixed: `x[n]=0.5*((n % 5)−2) + 0.25i*((n % 7)−3)`, where modulo is integer arithmetic before conversion; input is finite with positive and negative real/imaginary components. All-zero remains a *native test-only* fixture, not a fifth UI preset.

Status and announcement language

Unstarted is **not** "stage 0 completed". Example: "FFT ready: N=8, impulse input. No stages completed. Previewing stage 1, pair 3, lanes 4 and 5." After stepping: "Stage 2 of 3 complete; block width 4; pair 3 of 4; lanes 4 and 6; twiddle W4^0 = 1." Pair changes speak the inspector details without listing every lane. Keep one live region outside the replaceable SVG roots and restore focus to the originating control after rebuild.

9. Layout, routing and performance

Geometry and fitted view

Use rectangular complex formatting with a small measured icon scale. Construct overview and inspector nodes with each slot's widest reachable trace value, then set the current value; `set_data_values` never resizes. Compute each column width from library-measured node rectangles and each row from fixed lane anchors. Reserve the toolbar margin independently of content.

The driver at (−10000, −10000) must never influence the calculated viewport. Derive SVG canvas dimensions, any custom fit-to-content and scroll extents only from the visible lane and inspector rectangles. Explicitly test Reset zoom and changes between N=8 and N=16 while panned: no remote snap toward the driver.

Human legibility is a release requirement

At N=16, stage **S4** (zero-based stage index **3**) pairs lanes separated by eight rows. Test all 128 real routes in an actual browser. CDP can check topology and SVG attributes, but cannot judge crossing density, label occlusion or arrowhead clarity. Capture and manually inspect a **full-canvas N=16 screenshot** at the final stage, with pair 1 and a long-span pair selected; repeat at default zoom and one zoomed/panned view.

Pass only if arrowheads, lane labels, selected routes and node boundaries remain distinguishable. If not, **increase measured stage-column gaps** (and canvas width) as the first fallback, rerender and record the before/after screenshots. Avoid a private-layer overlay and avoid changing the public connector API merely to pass the first legibility check.

Performance expectations from bench-butterfly

For butterfly half-width 1–16, split-array WebAssembly `simd128` was up to approximately **2.5× slower**, and a whole FFT showed **no improvement**. All stages for UI N≤16 have half≤8, so the working assumption is *no SIMD benefit*. Absolute transform cost is negligible; focus measurement on SVG construction, path routing, accessibility and updates.

Record N=8 and N=16 browser scene-build time, edge routes and snapshot-change time. Continue with existing compiler-generated complex kernels. Fusing short stages or batching blocks is only a future, benchmark-led optimisation for larger transforms; do not add explicit intrinsics or change the public Butterfly API.

CASE	FFT LANES + DRIVER	ROUTES	BLOCK CALLS
N=8	32 + 1 = 33	48	7
N=16	80 + 1 = 81	128	15

10. Verification and acceptance tests

AREA	REQUIRED COVERAGE
Numerical oracle	N=2,4,8,16,32 native; independent O(N^2) DFT using sin/cos rather than reused twiddle or butterfly helpers; unscaled forward sign. Reject N=0,1,3 before bit reversal, without panic.
Fixtures	UI presets: impulse x[1], unit constant, bin-1 cosine, and stated modular mixed complex. All-zero native-only. Test cosine peaks in bins 1 and N−1 (amplitude N/2); inverse roundtrip non-UI.
Bit reversal	N=8 exact mapping, bijective/involutive for all supported sizes, single collected permutation and natural-order output.
Trace	N/2 records per stage, exact start-major/j-minor enumeration, pair disjointness, source indices from previous-stage state; scalar product, sum, difference cross-checked to buffer results.
Default pair	Impulse N=8 initial stage 1: first nonzero pair is pair index 2 (lanes 4,5); constant input: index 0; all-zero: index 0. Restart repeats the same rule.
Overview	N=8: 32 FFT nodes + 1 driver / 48 edges; N=16: 80 FFT nodes + 1 driver / 128 edges. Count by lane names or recorded NodeIds; selected pair focuses four proper edges and lane endpoints.
UI/CDP	panel-fft lazy init; native N/preset/pair selects; None/Some stages and unstarted announcement; unreachable marks; AX tree and keyboard order at N=16; zoom, fit excludes hidden driver; drag/view restoration.
Failure injection	Stage render fails after toolbar moves: driver rolled back. Pair failure restores HTML select; injected rollback failure enters repair state; Retry recovers; stale rebuild callbacks are ignored.
Manual screenshot	At N=16 S4 verify arrowheads, labels, long-span crossings, four focused routes at normal and zoomed/panned views; widen stage-column gaps and retest when illegible.
Rendering cost	Measure N=8/16 actual browser build and stage render times; expect SIMD no gain. Record SVG counts and ensure no browser console errors.

Tolerance

For small f64 fixtures, an illustrative absolute bound is `|got−want| ≤ 1e−12 × max(1, N, |want|)`, supplemented by an input-norm-sensitive bound for larger magnitudes. Pin the bound in native tests. Tests of three-decimal visual text are additional checks, never the sole numerical oracle.

Acceptance gate: no dependency on private Scene SVG content, no custom overlay, every forward N=8/N=16 fixture correct, deterministic pair selector, no silently advanced toolbar after failures, and visibly retained drag/zoom/focus across rebuilds; pass manual N=16 route legibility and AX-tree checks.

11. File plan and delivery order

PHASE	FILES AND DELIVERABLES
1 · Arithmetic	`demo-app/src/fft/algorithm.rs` and its unit tests: N≥2/nonfinite validation, bit reversal, full-block DIT, complete states and PairTrace metadata.
2 · Overview	`demo-app/src/fft/layout.rs`, `overview.rs`: real per-lane nodes, public Scene edges, measured positions excluding hidden driver, EdgeId table and highlighting.
3 · Inspector/UI	`demo-app/src/fft/inspector.rs`, `mod.rs`: Butterfly detail, widest-value construction, stage driver, pair selector, repair protocol, preservation of view and frame origin.
4 · Integration	`demo/panels/panel-fft.html` with labelled native N/preset/pair selects, `id="panel-fft"` and SVG IDs `fft-overview-diagram` / `fft-inspector-diagram`; add to `demo-app/src/lib.rs` (gallery and source-file list); `docs/demo/fft.md`.
5 · CDP	`cdp-integration-test/tests/cdp/fft.rs` and test registration: number and topology of edges, selected focus, cursor rollback, retry, input changes and accessibility / tab traversal. Manually inspect N=16 route screenshots before release.

Decisions now fixed

- **Topology:** separate per-lane nodes and standard directed connectors, including N=16. No conditional overlay or core API.
- **Navigation:** one completed stage per toolbar position, plus a separate pair selector; initial and restarted inspector show the deterministic first-informative stage-1 pair.
- **Numeric implementation:** negative-angle forward twiddles and one buffer call per block; scalar PairTrace fields derived before mutation and tested against the buffer output.
- **Error semantics:** a stage update that fails restores the stage driver (or enters explicit recovery); selection and data never silently diverge.
- **Release:** demo-only change; follow actual project release policy without rewriting historical commit-derived changelog entries.

Ready for Phase 1
This revision defines input validation, exact controls and presets, committed cursor/pair recovery, node-count scope, accessibility exposure and a manual N=16 route-legibility gate. Phase 1 can begin without any new svg-dom-graph public API.

Appendix A · N=8 impulse-stage reference

Canonical input: x=[0,1,0,0,0,0,0,0]. S0 is bit-reversed; every later state is fully computed after its named stage. Entries below are rounded for documentation; tests compare native floating-point values to a direct DFT.

LANE i	S0	S1 (m=2)	S2 (m=4)	S3 = X[i]
0	0	0	0	1
1	0	0	0	0.707−0.707i
2	0	0	0	−i
3	0	0	0	−0.707−0.707i
4	1	1	1	−1
5	0	1	1	−0.707+0.707i
6	0	0	1	+i
7	0	0	1	0.707+0.707i

At stage 3, pair index 1 (user-facing pair 2) has `start=0, j=1, upper=1, lower=5, a=0, b=1,` w=`(1−i)/√2`. The scalar product equals w; the result is +w in lane 1 and −w in lane 5.

Appendix B · API references (pinned at repository baseline)

R1 · `demo-common/src/complex/buffer.rs` — ComplexBuf stage kernel, positive roots, conj

R2 · `demo-common/src/complex/unit_tests/buffer.rs` — existing full DIT helper and DFT oracle

R3 · `src/scene/node/butterfly/mod.rs` — Butterfly frame, operands, results and dragging

R4 · `src/scene/connector/mod.rs` — add_edge_with and set_edge_focus

R5 · `src/scene/selection_toolbar/mod.rs` — driver mapping, callback lifecycle and safe reentry

R6 · `docs/library/complex-numbers.md` — ComplexNumber formats and fixed node sizing

R7 · `demo-app/src/butterfly.rs` — live inspector and widest-value construction

R8 · `demo-app/src/dijkstra/mod.rs` — step driver, status and focus preservation

R9 · `demo-app/src/lib.rs` — panel registration and file lists

R10 · `bench-butterfly/README.md` — benchmark harness and results context

Revision history: v2 replaced the unsupported overlay with true graph topology and specified stage/pair rollback. v3 closes the six remaining review findings: N≥2; hidden-driver count and fit; connector accessibility; four precise presets plus native selects; manual N=16 route screenshots; unstarted wording and committed-pair cache definition.